Tarea 4 — Gramática de MarkdownCheck

Lenguajes Formales y de Programación, Sección B+, Segundo Semestre 2026
Josué Eliú Castro Sosa — carné 202405110

1. Tabla de terminales

Estos son los tokens que usa mi gramática. Los que dicen Nuevo o Modificado son los que no están igual que en la tabla 4.5 del enunciado.

Token	Qué es	Ejemplo	¿Cambio?
NUMERAL	De 1 a 6 # al inicio de línea, seguidos de un espacio. Marca un encabezado.	##	Igual
VIÑETA	Un -, * o + al inicio de línea (después de la sangría), seguido de un espacio. Marca un ítem de lista.	-	Igual
INDENT	Token invisible: aparece cuando un ítem queda un nivel más adentro (2 espacios más) que el anterior.	(sin texto)	Igual
DEDENT	Token invisible: aparece cuando se sale de un nivel. Si se suben dos niveles de golpe salen dos DEDENT.	(sin texto)	Igual
CITA	Un solo símbolo >. Un >> son dos CITA seguidos, y > > da lo mismo.	>	Modificado
CERCA_CODIGO	Tres acentos graves al inicio de línea. Abre y cierra un bloque de código.	```	Igual
LENGUAJE	Palabra pegada a la cerca de apertura que dice el lenguaje del bloque.	python	Nuevo
TEXTO_LITERAL	Todo lo que hay dentro de un bloque de código o de un código en línea, tal cual, en un solo token.	x = 2 ** 3	Nuevo en la tabla (ya aparecía en la gramática 4.6)
REGLA_HORIZONTAL	Tres o más guiones solos en una línea.	---	Igual
NEGRITA	Doble asterisco. Abre o cierra una negrita.	**	Igual
CURSIVA	Un asterisco que no está en posición de viñeta. Abre o cierra una cursiva.	*	Igual
CODIGO_INLINE	Un acento grave simple. Abre o cierra un código en línea.	`	Igual
TEXTO	Texto normal que no es ninguno de los símbolos anteriores.	Notas	Igual
SALTO_PARRAFO	Una línea en blanco. Ella sola es el token completo (no lleva FIN_LINEA aparte).	(línea vacía)	Igual
FIN_LINEA	El final de una línea con contenido.	(salto de línea)	Nuevo
EOF	El final del archivo.	(sin texto)	Nuevo

Por qué agregué o cambié estos:

- **FIN_LINEA:** sin él la gramática no sabe dónde termina un párrafo de varias líneas, un ítem de lista o un encabezado. Es el token de fin de línea que sugiere la tarea.
- **EOF:** para que el documento tenga un final claro y el parser sepa cuándo parar.

- **TEXTO_LITERAL:** en la gramática del enunciado aparece pero no estaba en la tabla de tokens, así que lo declaro para que ambas cosas coincidan.
- **LENGUAJE:** el bloque de código puede traer la palabra python pegada a la cerca, y la necesito para poner class="language-python" en el HTML. Es opcional en la gramática.
- **CITA (modificado):** en la tabla 4.5 un solo token CITA llevaba la cantidad de símbolos. Yo uso un token por cada >, así la profundidad de la cita sale sola de la recursión de la gramática y >> y > > dan los mismos tokens.

2. Gramática en BNF

Los no terminales van entre < > y los tokens en MAYÚSCULAS. El símbolo inicial es **<documento>**. ε significa que la regla puede quedar vacía.

```
<documento> ::= <bloques> EOF
<bloques> ::= <bloque> <bloques>
           | ε
<bloque> ::= <encabezado>
          | <parrafo>
          | <lista>
          | <cita>
          | <bloque_codigo>
          | <regla>
          | SALTO_PARRAFO
<encabezado> ::= NUMERAL <en_linea> FIN_LINEA
<regla> ::= REGLA_HORIZONTAL FIN_LINEA
<parrafo> ::= <linea_texto> <parrafo_resto>
<parrafo_resto> ::= <linea_texto> <parrafo_resto>
                  | ε
<linea_texto> ::= <elemento> <en_linea> FIN_LINEA
<lista> ::= <item_lista> <lista_resto>
<lista_resto> ::= <item_lista> <lista_resto>
                | ε
<item_lista> ::= VIÑETA <en_linea> FIN_LINEA <sublista_opt>
<sublista_opt> ::= INDENT <lista> DEDENT
                  | ε
<cita> ::= <linea_cita> <cita_resto>
<cita_resto> ::= <linea_cita> <cita_resto>
               | ε
<linea_cita> ::= CITA <cuerpo_cita>
<cuerpo_cita> ::= CITA <cuerpo_cita>
                | <en_linea> FIN_LINEA
<bloque_codigo> ::= CERCA_CODIGO <lenguaje_opt> FIN_LINEA <contenido_cod> CERCA_CODIGO FIN_LINEA
<lenguaje_opt> ::= LENGUAJE
                  | ε
<contenido_cod> ::= TEXTO_LITERAL
                   | ε
<en_linea> ::= <elemento> <en_linea>
             | ε
<elemento> ::= <negrita>
             | <cursiva>
             | <codigo_inline>
             | TEXTO
<negrita> ::= NEGRITA <en_negrita> NEGRITA
<en_negrita> ::= <elem_neg> <en_negrita>
               | ε
<elem_neg> ::= <cursiva>
             | <codigo_inline>
             | TEXTO
<cursiva> ::= CURSIVA <en_cursiva> CURSIVA
<en_cursiva> ::= <elem_curs> <en_cursiva>
               | ε
<elem_curs> ::= <negrita>
              | <codigo_inline>
```

```
            | TEXTO
<codigo_inline> ::= CODIGO_INLINE TEXTO_LITERAL CODIGO_INLINE
```

3. Ajustes para análisis descendente recursivo

El parser descendente recursivo decide qué regla usar mirando solo el siguiente token. Para eso la gramática no puede tener reglas que empiecen llamándose a sí mismas, ni dos alternativas que empiecen igual. Revisé la 4.6 regla por regla y esto fue lo que encontré y cambié:

#	Problema en la 4.6	Qué hice	Qué resuelve
1	**Recursión por la izquierda:** la busqué en todas las reglas y no hay. Las repeticiones (documento, lista, en_linea) se llaman a sí mismas al final de la regla, no al inicio.	Dejé todas las repeticiones al final de la regla.	Una regla que empieza llamándose a sí misma haría que la función se llamara infinitamente sin avanzar. Aquí eso no pasa.
2	**<lista>:** las dos alternativas empezaban con <item_lista>.	La dividí en <lista> ::= <item_lista> <lista_resto>. <lista_resto> decide si sigue otro ítem o se acaba.	El parser ya no tiene que adivinar: consume el primer ítem y luego mira si viene otra VIÑETA.
3	**<item_lista>:** las dos alternativas empezaban con VIÑETA <en_linea>.	Un solo camino: VIÑETA, texto, FIN_LINEA y luego <sublista_opt>, que decide si hay sublista o no.	La decisión se toma con INDENT, que es distinto a todo lo demás.
4	**<cita>:** las dos alternativas empezaban con CITA <bloque>.	La dividí en <cita>, <cita_resto> y <linea_cita>.	Es la misma idea que con la lista.
5	**El párrafo** era de una sola línea, y como <en_linea> puede ser vacío, un párrafo podía ser nada.	Ahora un párrafo es una o más <linea_texto>, y cada línea empieza con un elemento y termina en FIN_LINEA.	La tarea pide párrafos de una o más líneas, y así un párrafo nunca queda vacío.
6	**Negrita y cursiva:** dentro de **...** el parser ve otro ** y no sabe si es el cierre o una negrita nueva.	Dentro de la negrita no se permite NEGRITA directa (<en_negrita>), y dentro de la cursiva no se permite CURSIVA directa (<en_cursiva>).	Si el parser está dentro de una negrita y ve NEGRITA, solo puede ser el cierre.
7	**Citas con bloques adentro:** la 4.6 hacía CITA <bloque>, o sea que una cita podía contener listas, encabezados, etc.	Lo simplifiqué: una línea de cita lleva texto en línea, y los niveles salen de repetir CITA.	Es lo que pide la tarea (> y >>). Es una simplificación a propósito.
8	**Faltaban piezas:** FIN_LINEA, EOF, la línea en blanco como bloque y la cerca de cierre con su salto de línea.	Agregué FIN_LINEA y EOF, puse SALTO_PARRAFO como bloque y completé <bloque_codigo>.	Sin eso, textos válidos (por ejemplo con líneas en blanco) no se podían reconocer.

4. Conjuntos PRIMERO y SIGUIENTE

PRIMERO de un no terminal son los tokens con los que puede empezar. SIGUIENTE son los tokens que pueden venir justo después de él. Para no repetir listas largas uso dos abreviaturas:

- **IL** = NEGRITA, CURSIVA, CODIGO_INLINE, TEXTO.
- **IB** = IL más NUMERAL, VIÑETA, CITA, CERCA_CODIGO, REGLA_HORIZONTAL, SALTO_PARRAFO.
- ε en PRIMERO quiere decir que ese no terminal puede quedar vacío. $ es el fin de la entrada.

No terminal	PRIMERO	SIGUIENTE
<documento>	IB, EOF	$
<bloques>	IB, ε	EOF
<bloque>	IB	IB, EOF

No terminal	PRIMERO	SIGUIENTE
<encabezado>	NUMERAL	IB, EOF
<regla>	REGLA_HORIZONTAL	IB, EOF
<parrafo>	IL	IB, EOF
<parrafo_resto>	IL, ε	IB, EOF
<linea_texto>	IL	IB, EOF
<lista>	VIÑETA	IB, DEDENT, EOF
<lista_resto>	VIÑETA, ε	IB, DEDENT, EOF
<item_lista>	VIÑETA	IB, DEDENT, EOF
<sublista_opt>	INDENT, ε	IB, DEDENT, EOF
<cita>	CITA	IB, EOF
<cita_resto>	CITA, ε	IB, EOF
<linea_cita>	CITA	IB, EOF
<cuerpo_cita>	IL, CITA, FIN_LINEA	IB, EOF
<bloque_codigo>	CERCA_CODIGO	IB, EOF
<lenguaje_opt>	LENGUAJE, ε	FIN_LINEA
<contenido_cod>	TEXTO_LITERAL, ε	CERCA_CODIGO
<en_linea>	IL, ε	FIN_LINEA
<elemento>	IL	IL, FIN_LINEA
<negrita>	NEGRITA	IL, FIN_LINEA
<en_negrita>	CODIGO_INLINE, CURSIVA, TEXTO, ε	NEGRITA
<elem_neg>	CODIGO_INLINE, CURSIVA, TEXTO	IL
<cursiva>	CURSIVA	IL, FIN_LINEA
<en_cursiva>	CODIGO_INLINE, NEGRITA, TEXTO, ε	CURSIVA
<elem_curs>	CODIGO_INLINE, NEGRITA, TEXTO	IL
<codigo_inline>	CODIGO_INLINE	IL, FIN_LINEA

Conflictos y cómo se resuelven

Encontré tres conflictos, todos del mismo tipo: <parrafo_resto>, <lista_resto> y <cita_resto>. Ahí el parser puede seguir con otra línea (u otro ítem) del mismo elemento, o dar por terminado el elemento y empezar un bloque nuevo, porque lo que viene después empieza con el mismo token que una línea más. Es el mismo problema del "else colgante".

- **Regla que aplica el parser:** si el token siguiente puede continuar el elemento actual, continúa (toma la alternativa que no es vacía). Solo termina cuando ve algo que no puede continuarlo: una línea en blanco, un DEDENT, el EOF o el inicio de otro tipo de bloque.
- **Por qué está bien:** en Markdown dos líneas de texto seguidas son el mismo párrafo, dos ítems seguidos son la misma lista y dos líneas con > seguidas son la misma cita. Para separarlos hay que dejar una línea en blanco, y esa línea es un token que no puede continuar nada.

5. Árboles de derivación

Los cuadros grises son no terminales y los de borde grueso son tokens. Como los árboles son grandes los partí en varias imágenes: un cuadro amarillo punteado significa que ese pedazo se dibuja aparte.

Sobre la sangría de la Cadena 1: en el PDF de la tarea no se distingue bien, así que la tomé como 0, 2 y 4 espacios (2 por nivel, como en el proyecto). Son tres niveles: Lexico, AFD y Estados con cursiva.

Cadena 1

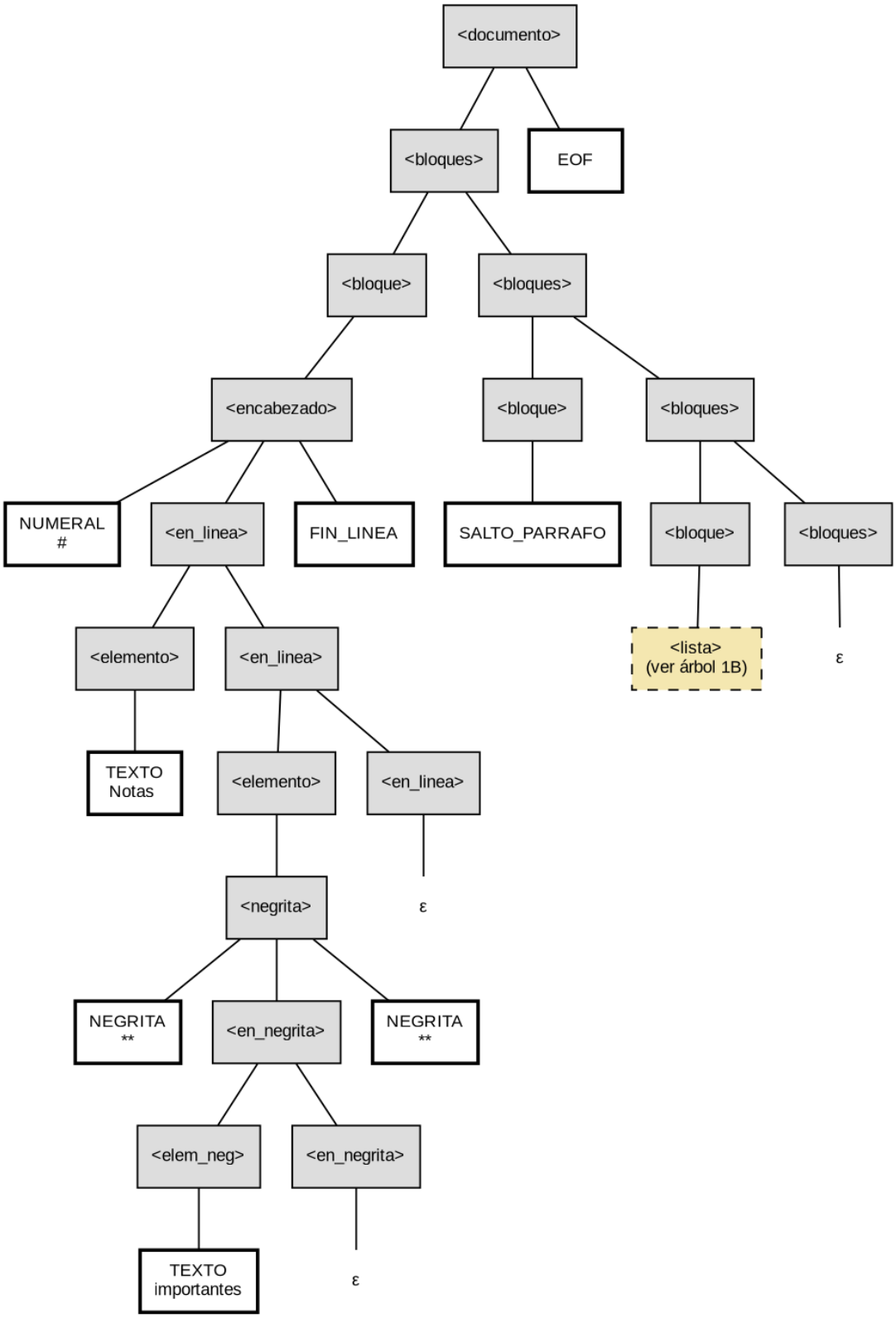
```
# Notas **importantes**

- Lexico
  - AFD
    - Estados con *cursiva*
- Sintactico
```

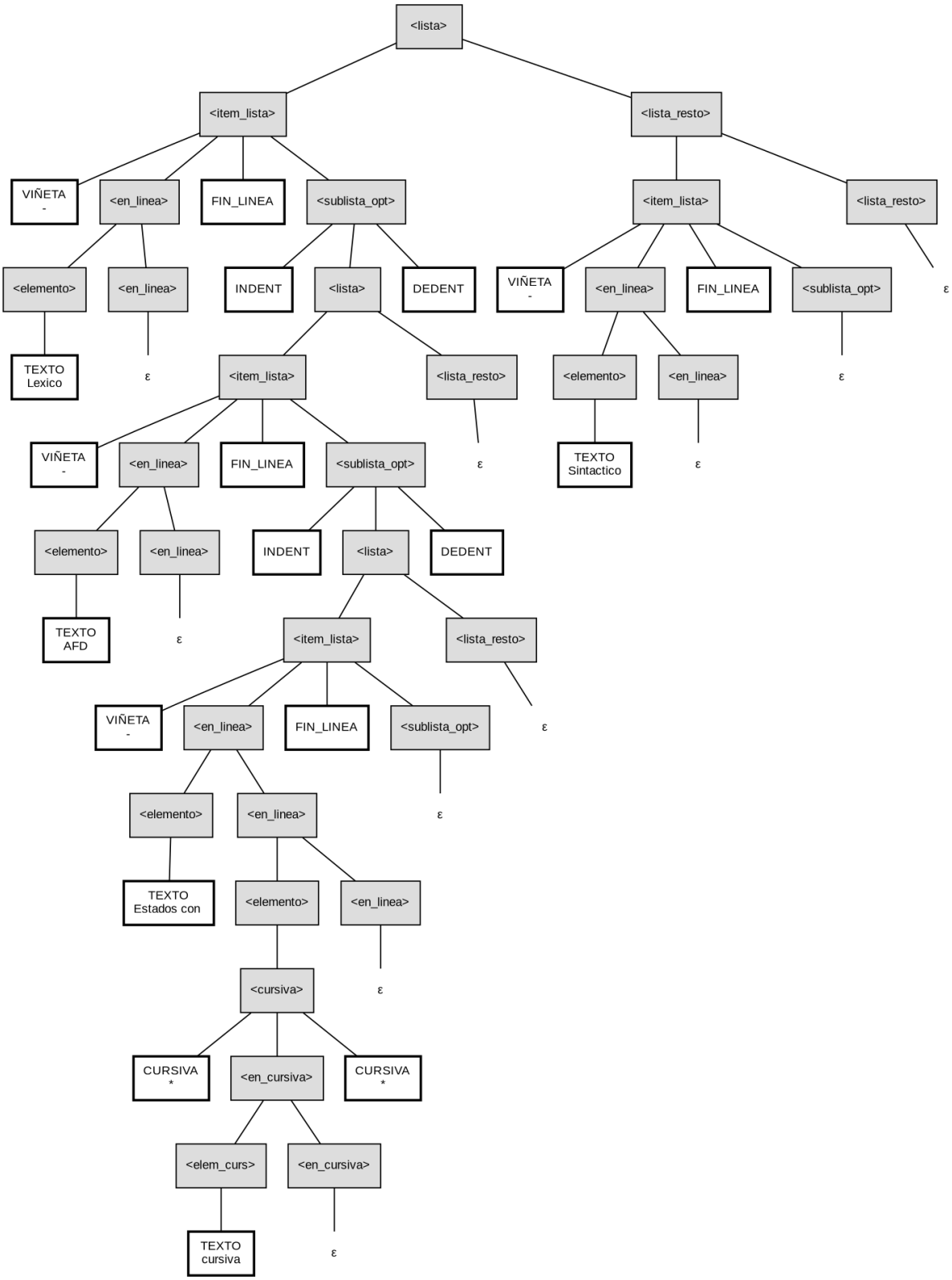
Secuencia de tokens:

Línea	**Tokens**
1	NUMERAL(#) TEXTO(Notas) NEGRITA(**) TEXTO(importantes) NEGRITA(**) FIN_LINEA
2 (vacía)	SALTO_PARRAFO
3	VIÑETA(-) TEXTO(Lexico) FIN_LINEA
4	INDENT VIÑETA(-) TEXTO(AFD) FIN_LINEA
5	INDENT VIÑETA(-) TEXTO(Estados con) CURSIVA(*) TEXTO(cursiva) CURSIVA(*) FIN_LINEA
6	DEDENT DEDENT VIÑETA(-) TEXTO(Sintactico) FIN_LINEA
Final	EOF

Árbol 1A: esqueleto del documento y el encabezado



Árbol 1B: la lista (tres niveles)



Cadena 2

````
> Una cita con `codigo`
>> Una cita anidada

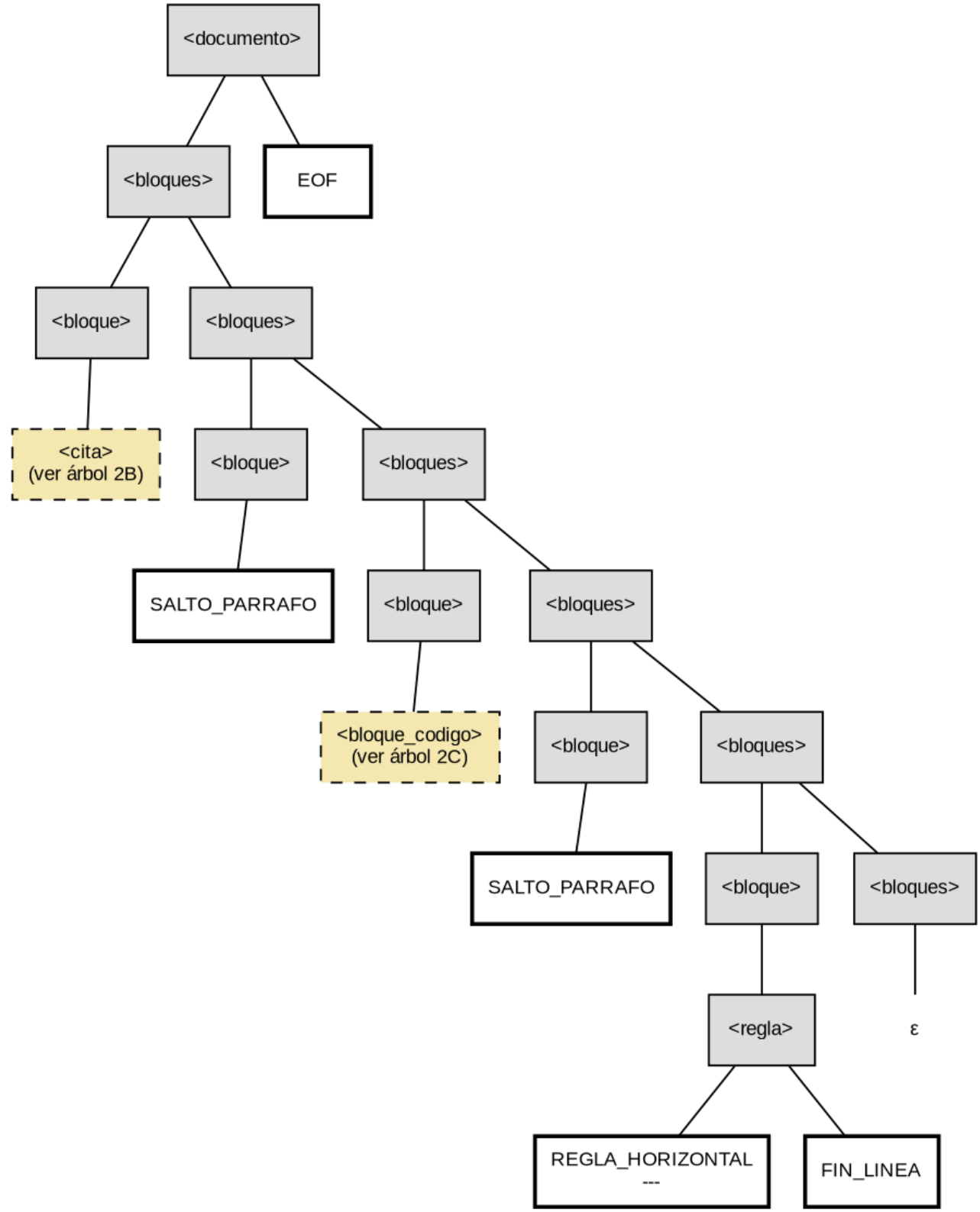
```python
x = 2 ** 3
```

---
````

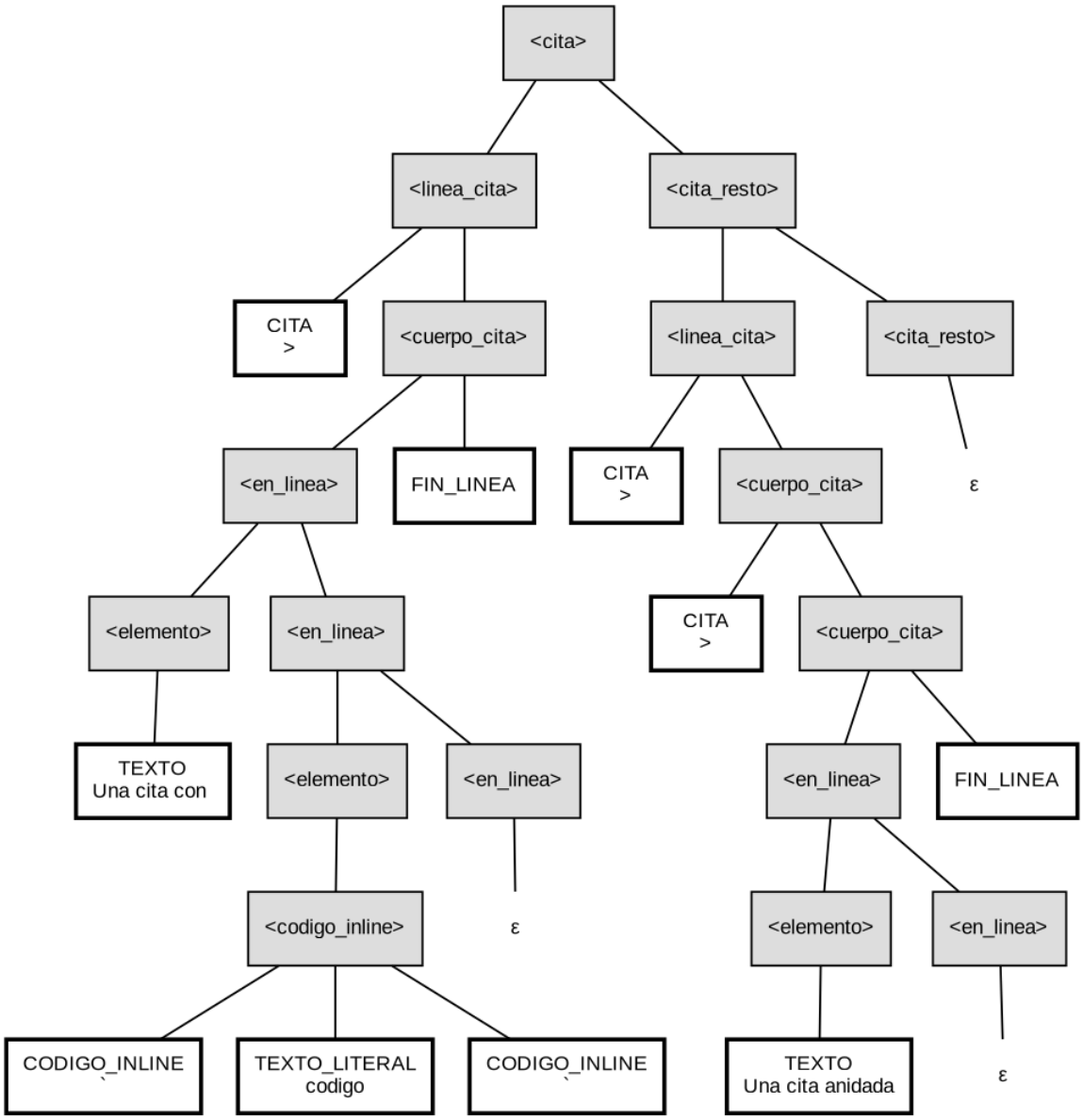
Secuencia de tokens:

Línea	**Tokens**
1	CITA(>) TEXTO(Una cita con) CODIGO_INLINE(`) TEXTO_LITERAL(codigo) CODIGO_INLINE(`) FIN_LINEA
2	CITA(>) CITA(>) TEXTO(Una cita anidada) FIN_LINEA
3 (vacía)	SALTO_PARRAFO
4 a 6	CERCA_CODIGO(```) LENGUAJE(python) FIN_LINEA TEXTO_LITERAL(x = 2 ** 3) CERCA_CODIGO(```) FIN_LINEA
7 (vacía)	SALTO_PARRAFO
8	REGLA_HORIZONTAL(---) FIN_LINEA
Final	EOF

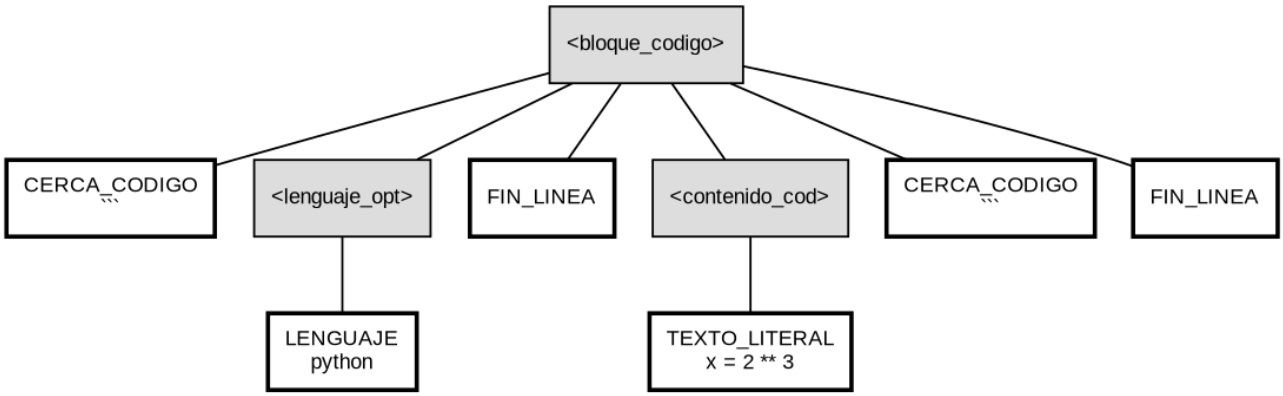
Árbol 2A: esqueleto del documento



Árbol 2B: la cita



Árbol 2C: el bloque de código



6. Cadena inválida

```
Esto es **importante
```

Es una negrita que se abre y nunca se cierra.

Tokens: TEXTO(Esto es) NEGRITA(**) TEXTO(importante) FIN_LINEA EOF

Dónde falla:

- <documento> → <bloques> EOF. El primer token es TEXTO, así que el bloque es un <parrafo> y su primera línea es una <linea_texto>.
- <linea_texto> toma un <elemento> (TEXTO(Esto es)). Luego <en_linea> ve NEGRITA y toma otro <elemento>, que es una <negrita>.
- <negrita> consume el NEGRITA de apertura y entra a <en_negrita>, que consume TEXTO(importante) sin problema.
- **Aquí falla:** <en_negrita> mira el siguiente token y es FIN_LINEA. Dentro de una negrita solo puede venir TEXTO, CURSIVA, CODIGO_INLINE o el NEGRITA que la cierra, y FIN_LINEA no es ninguno.

Por qué se rechaza: la gramática exige que toda negrita tenga su cierre antes de que termine la línea. Aquí la línea se acaba con la negrita todavía abierta y no hay ninguna regla que lo permita.